

2026夏季训练营 CUDA 方向项目

项目阶段信息和简介

关于项目阶段

项目阶段相比较于课程阶段的作业会更加 **open-ended** (开放式题目)，不会像作业一样提供测例和标准答案。项目阶段的目标不仅仅是更全面的检验和提升学员对 CUDA 编程的掌握能力，同时还是更全面的检验包括问题分析、方案与系统设计、工具使用和文档能力在内的**综合能力**。能优秀完成项目阶段的学员能具备工业级的开发能力。因此项目阶段的评审会综合和全面的考察和评判。

项目选题是什么？

项目阶段有多个选题，每个选题的方向和内容可能不尽相同，学员可以根据自己的兴趣进行选择。每位学员需**至少选择一个选题**且**欢迎**有兴趣和能力的同学同时**完成多个选题**。如果学员对选题有自己的想法，也欢迎与导师讨论。

项目阶段其他重要信息

项目阶段时间：8.18 - 9.20

项目提交方式：大部分选题与专业阶段作业相同，在官网提交 fork 仓库的链接与 commit 链接。具体提交方式各个选题可能有所不同，具体见项目选题要求。

项目代码要求

1. 最终提交需整理代码：命名方式统一（不限定具体某种风格，全程序统一即可）、代码格式化（无具体某种格式要求，但可以默认使用默认配置的 clang-format）、无测试代码并且关键函数和代码块进行适当的注释。**但与其他项目交叉的情况需遵循其他项目的规范；**
2. 取决于具体选题，大体上是以总体性能为评判标准，因此代码的某些功能在**适宜的情况下**可以在 CPU 上完成，但必须仍有较为显著的部分在 GPU 上完成。**总体性能需超越 CPU 同级别实现；**
3. 如有需要，可以使用合适的 CUDA 官方库。
4. 如果选题的提交地址为 Learning-CUDA 仓库，则需要 fork Learning-CUDA 的 `2026-summer-project` 分支，并在其下对应的选题文件夹中放置自己的代码，即放置路径为 `project` 分支下 `/<选题>/<你的ID>/`。提交方式为 PR。例如 ID 为 lzm 并选做了 `path_finding` 的选题，则其代码提交地为 `/path_finding/lzm/`。

如有任何疑问或不确定的地方，请问导师或助教！不要等到最后提交反馈后才发现有不合规的地方！

项目选题

需注意，各个选题的基础难度各不相同，在晋级评定时会有影响。同时，是否完成额外目标或进阶优化也会直接影响评定。

选题描述可能会有部分细节没有罗列，具体细节问题请询问导师和助教！

一. BF16 核函数优化（CUDA + 训练）

简介：此选题是 CUDA 与训练框架方向的交叉选题，鼓励同时学习了 CUDA 与训练方向的学员选择。如果没有学习过训练方向但有兴趣做的也欢迎选择。

当前 InfiniTrain 中有诸多支持 GPT2 和 Llama3 进行 BF16 类型训练的核函数。但当前的 BF16 情况下的整体性能需要提升。学员需要运行 InfiniTrain、分析当前 BF16 情况下的性能瓶颈、进行核函数优化，最终以提升 BF16 下核函数本身以及端到端性能为评判标准，尤其是 BinaryBackward，且 FP32 情况下的性能不能有明显下降。

性能对照基准：PyTorch。

更多详情可以参考 [📄【训练方向 2026 夏季训练营】项目选题文档](#) 中的**第二与第三部分**，以及 InfiniTrain 仓库链接与 README。

需提交内容：

- 优化好的核函数以及附属/相关改动的代码。
 - 提交地址：[InfiniTrain 仓库](#)
 - 提交要求：
 - i. 需遵循 InfiniTrain 开发规范，具体可见训练方向的选题文档；
 - ii. PR 提交命名：**【训练营】BF16 Kernel 优化**；
 - iii. 如果需要端到端验证性能，相关参数和数据集文件会传到训练营服务器共享文件夹里，具体可参考训练方向选题文档里的说明。
- 总结报告。报告需包括：
 - 问题描述
 - 当前（即优化前）的性能记录和分析
 - 详细阐述优化方法，欢迎包含自己的优化历程和记录，开发中发现的问题等（取决于质量可以加分）
 - 最终优化后的性能记录和分析
 - 未来可继续提升的地方（可选）
 - 包含 ncu 使用和分析的加分

二. MXFP8 / NVFP4 低精度模拟与反量化（CUDA）

凭借你在 InfiniTensor 训练营的经历和人工智能与并行计算方面的出色成绩，你受雇于一家领先的 AI 基础设施公司，负责为下一代大模型推理引擎开发低精度权重加载模块。公司正在评估 MXFP8、NVFP4 等格式在模型压缩和推理加速中的价值，但现实问题是：测试环境并不能提供 Hopper、Blackwell 或更新架构的 GPU，而目前手中的 GPU 只支持常规 CUDA kernel。

你需要实现一套不依赖新硬件指令的低精度格式软件模拟与反量化程序：在普通 CUDA GPU 上完成低精度编码、打包、缩放、解包、反量化与误差评估；如果有更高架构硬件，则可以额外对比 CUDA Math API、Transformer Engine 或硬件加速路径。

领域知识简介

FP8 常见格式包括 E4M3 和 E5M2。MXFP8 是一种 microscaling FP8 格式。NVFP4 使用 E2M1 的 4 bit 数值，并结合分层缩放恢复高精度值。具体介绍可参考本方向的相关课程和特定大咖课。更多详细信息需要各位学员自行调研。

任务内容

开发一个 CUDA 程序，实现低精度浮点格式的软件模拟与反量化，须支持 MXFP8 和 NVFP4。普通 FP8 不作为基础验收要求，可作为调试参考、误差对照或扩展功能。程序需要完成以下功能：

1. 读取 FP32 或 FP16 输入矩阵；
2. 根据配置将输入矩阵量化为指定低精度格式，并按真实 bit 宽度打包存储；
3. 保存低精度数据和缩放因子；
4. 使用 CUDA kernel 将低精度数据反量化为 FP16、BF16 或 FP32；
5. 输出误差统计和性能日志。

输入文件定义

1. 张量数据文件

代码块

```
1  [header]
2  num_rows: int64           # 矩阵行数
3  num_cols: int64          # 矩阵列数
4  dtype: string             # 输入数据类型: fp32 或 fp16
5
6  [data]
7  values: dtype[num_rows * num_cols] # 按行主序存储
```

2. 量化参数文件

代码块

```
1  format = "mxfp8"           # mxfp8 / nvfp4
2  block_size = 32            # mxfp8 默认为 32, nvfp4 默认为 16
3  scale_mode = "block"       # tensor / block
4  output_type = "fp16"       # fp16 / bf16 / fp32
5  rounding = "nearest"       # nearest / stochastic
6  target_gpu = "T4"          # 目标 GPU 型号, 仅用于报告说明
```

输出文件定义

- **低精度权重文件**：二进制文件，包含 packed data、scale 数组和必要的 header 信息。4 bit 格式必须做到每个字节存储两个元素，不能用 `uint8` 存一个 4 bit 元素。
- **反量化后的张量**：二进制文件，按行主序存储，数据类型由 `output\_type` 指定。
- **误差与性能日志**：包含最大绝对误差、平均绝对误差（MAE）、均方误差（MSE）、压缩率、量化 kernel 时间、反量化 kernel 时间、有效内存带宽（GB/s）。

要求

1. 功能正确性：

- 正确实现 MXFP8 数据表示、block scaling 和反量化流程；
- 正确实现 NVFP4 的数值编码、元素局部缩放、全局缩放、打包、解包与反量化规则；
- 正确实现 per-tensor scaling 和 block-wise scaling；
- 对随机矩阵、正态分布矩阵和含异常值矩阵分别输出误差统计。

2. 硬件要求：

- 基础版本不得依赖 Hopper、Blackwell 或 Ampere 以上架构特性；
- 基础版本不得要求硬件原生支持 FP8 / FP4 Tensor Core；
- 如有支持 FP8 / FP4 的 GPU，可额外实现硬件或库路径作为加分项，但不能作为基础验收前提。

3. 4 bit 打包与内存访问：

- 4 bit 格式必须实现 packed load / packed store；

4. 平台适配：默认需在英伟达平台上支持。每在一款国产平台上支持，额外加分。

需提交内容：

- 整个完成上述功能的程序（包含测试）
 - 提交地址：[Learning-CUDA 本季度项目分支 \(2026-summer-project\)](#)
 - 提交要求：参照本文档开头**项目阶段信息和简介-项目代码要求**中的**第四条**
- 总结报告。报告需至少包括：
 - 详细阐述低精度格式、缩放策略、打包布局和反量化公式

- 详细阐述实现思路以及优化方法，欢迎包含自己的优化历程和记录，开发中发现的问题等（取决于质量可以加分）
 - 最终的误差指标、压缩率、性能指标和分析
 - 明确说明哪些实现为软件模拟，哪些实现依赖特定 GPU 架构或第三方库
 - 未来可继续提升的地方（可选）
 - 包含 `ncu` 和/或 `nsys` 使用和分析的加分
-

三. Hadamard 变换加速（CUDA）

最新研究表明，在量化前对激活进行随机旋转（如 Hadamard 变换）可以有效抑制异常值，提升低比特量化精度——这是 QuaRot、SpinQuant、FlashAttention-3 等前沿工作的核心思想。然而，Hadamard 变换本身引入了额外计算开销，可能抵消量化带来的收益。

公司要求你实现一个加速的 Hadamard 变换核，将旋转开销降至最低，使得 FP8/INT4 量化在保持精度的同时真正加速，并且建议优先探索利用 Tensor Core。

领域知识简介

Hadamard 变换的价值：神经网络激活中存在异常值，导致量化误差增大。Hadamard 变换是一种正交变换，可以将异常值能量“扩散”到多个维度，降低单个维度的动态范围。QuaRot 和 SpinQuant 证明，在注意力计算前后插入 Hadamard 变换，可使 INT4/FP8 量化精度接近 FP16。更多详细信息需要各位学员自行调研。

任务内容

实现一个快速 Hadamard 变换核，支持多种输入尺寸，并与量化算子融合，最好能基于 Tensor Core。

输入文件定义

1. 一系列输入张量的形状与类型。输入张量：

`[batch_size, seq_len, num_heads, head_dim]`，FP16 或 BF16

- Hadamard 矩阵大小：`head_dim`（必须是 2 的幂次，如 64、128、256）

输出文件定义

- **性能日志**：核函数执行时间（ms）。

要求

1. 功能正确性：

- 对于 FP16，与参考实现（如 `fast_hadamard_transform` 库）的绝对误差小于 `1e-2`。

- 对于 BF16，绝对误差小于 $5e-2$ 。
- 当融合量化时，需验证量化后的结果与先变换后量化的结果一致。

2. Tensor Core：可使用 Tensor Core 进行加速。

- a. **进阶**：和未使用 Tensor Core 加速的实现进行对比（Note：实现算法不一定相同）。

3. 平台适配：默认需在英伟达平台上支持。每在一款国产平台上支持，额外加分。

需提交内容：

- 整个完成上述功能的程序（包含测试）
 - 提交地址：[Learning-CUDA 本季度项目分支 \(2026-summer-project\)](#)
 - 提交要求：参照本文档开头**项目阶段信息和简介-项目代码要求**中的**第四条**
- 总结报告。报告需包括：
 - 详细阐述实现思路以及优化方法，欢迎包含自己的优化历程和记录，开发中发现的问题等（取决于质量可以加分）
 - 最终的性能指标和分析
 - 未来可继续提升的地方（可选）
 - 包含 ncu 和/或 nsys 使用和分析的加分

四. 生命序列比对（CUDA）

任务背景

鉴于你有 InfiniTensor 训练营的经历，并且在人工智能与并行计算方面的成绩出众，你被秘密邀请于京西航天港登上第一艘装备了我国曲速一引擎的“寻星号”科研探索船。登舰之后，舰长对你简单地讲述了任务并让你为之尽快做好准备。尽管舰长言辞隐晦并对你的诸多问题不予理睬，但你已然猜出此次任务与一百年前在“播种计划”中于太微左垣二星域失踪的“蒲公英号”有关。感受过曲速一引擎的伟力之后，你们成功到达了目标星域。

任务团队采集到了未知生物的 DNA 样本（实际可能由未知的碱基类似物组成，但为简化视为 A、T、C、G）。为了解该生物与地球生命的关联，需要将海量测序得到的未知 DNA 片段（reads）与已知的地球生物基因数据库以及此前发现的其他外星物种基因库进行比对。

然而，这些基因库异常庞大，且测序仪在行星表面每秒产生数百万条 reads。你需要利用星舰上的 GPU 计算集群开发一个超高速比对程序，实时处理从探测器传来的数据流，迅速定位每条 reads 在参考基因库中的最佳匹配位置，并标识出高度相似或完全未知的序列。无需多言，这项任务至关重要。

领域知识简介

在基因测序中，测序仪产生大量短片段（称为 reads），每个 read 是 DNA 序列的一段（由 A、T、C、G 组成）。这些 reads 需要比对到参考基因组上，以确定每个 read 的来源位置。参考基因组是已知的

完整 DNA 序列（如人类基因组约 30 亿碱基），通常按染色体分成多个文件。比对结果用于变异检测、基因表达分析等。由于 reads 数量可达数亿甚至数十亿，计算量巨大，需要 GPU 加速。

任务内容

开发一个 CUDA 程序，能够将大量 reads 快速比对到参考基因组上。

输入文件定义

1. 给定一个**参考基因组文件（FASTA 格式）**包含多条染色体/序列，每条以 `>` 开头的行为序列名，后续行为碱基序列（可跨多行）。示例：

代码块

```
1  >chr1
2  ATCGATCGATCGATCG...
3  >chr2
4  GCTAGCTAGCTAGCTA...
```

以及

2. **测序 reads 文件（FASTQ 格式）**：每个 read 由四行组成：

- 第 1 行以 `@` 开头，为 read 标识符；
- 第 2 行为碱基序列；
- 第 3 行以 `+` 开头（可选后续描述）；
- 第 4 行为质量分数（与碱基一一对应，表示测序准确度）。

示例：

代码块

```
1  @read1
2  ATCGATCG
3  +
4  IIIIIIII
5  @read2
6  GCTAGCTA
7  +
8  IIIIIIII
```

输出文件定义

比对结果文件。每行包含：

- read 名称

- 参考序列名
- 起始位置（0-based 坐标）
- 比对得分（如匹配数-错配数-空位数）。若无法比对，标记为 "unknown_origin"

示例：

代码块

```
1 read1 chr1 0 8
2 read2 chr2 0 8
3 read3 unknown_origin
```

要求

1. 参考基因组文件（FASTA 格式）可以忽略第四行，只使用碱基序列进行比对（即认为所有碱基同等可靠）。**进阶：**如果使用第四行，使用正确可以额外加分；
2. 程序需支持参考基因组文件中至少有一百条序列，每条序列至少需有一亿的长度（刚开始测试可以先从短序列开始）；
3. 程序需支持**测序 reads 文件**中 reads 数量至少十万条，每条 read 至少需有一百的长度（但可以默认不会超过两百的长度）；
4. 得分参数：匹配+2，错配-1，空位罚分-1（线性空位罚分）。
5. 默认需在英伟达平台上支持。每在一款国产平台上支持，额外加分。

需提交内容：

- 整个完成上述功能的程序（包含测试）
 - 提交地址：[Learning-CUDA 本季度项目分支 \(2026-summer-project\)](#)
 - 提交要求：参照本文档开头**项目阶段信息和简介-项目代码要求中的第四条**
- 总结报告。报告需包括：
 - 详细阐述实现思路以及优化方法，欢迎包含自己的优化历程和记录，开发中发现的问题等（取决于质量可以加分）
 - 最终的性能指标和分析
 - 未来可继续提升的地方（可选）
 - 包含 ncu 和/或 nsys 使用和分析的加分

五. 金融衍生品定价与风险估计（CUDA）

任务背景

鉴于你有 InfiniTensor 训练营的经历，并且在人工智能与并行计算方面的成绩出众，你被一家量化交易公司邀请加入高性能计算团队。公司每天需要为大量复杂衍生品重新估值，以控制风险敞口和交易保证金。传统 CPU 实现虽然精度可靠，但面对数千万条随机路径和多种路径依赖期权时，计算速度无法满足实时风控需求。

你注意到，期权路径之间基本相互独立，非常适合 GPU 并行。你的任务是开发一个 CUDA 加速的衍生品定价程序，使其能够在普通 GPU 上快速计算欧式期权、亚式期权或障碍期权等合约的价格，并输出可解释的误差或收敛指标。

领域知识简介

金融衍生品的价格取决于标的资产价格、执行价格、利率、波动率、到期时间和收益函数等因素。对简单欧式期权，可以使用 Black-Scholes 解析公式作为正确性参考。在该模型中，标的资产价格 S_t 可近似为几何布朗运动：

|

$$dS = rSdt + \sigma * SdW$$

其中 S 是资产价格， r 是无风险利率， σ 是波动率， dW 是布朗运动增量。

对于没有解析解或解析解复杂的路径依赖期权，通常需要使用数值方法近似求解。例如可以模拟大量价格路径，在每条路径上计算到期收益，再对收益折现取平均。

任务内容

开发一个 CUDA 程序，实现衍生品定价与风险指标估计。程序需要至少支持以下功能：

- 1. 解析期权参数、市场参数和模拟参数；
- 2. 根据自行选择的方法，在 GPU 上完成主要计算；
- 3. 计算期权价格，并至少输出一种误差、收敛或置信度指标；
- 4. 对欧式期权提供解析解或高精度 CPU 结果作为参考；
- 5. 输出结果文件和性能日志。

输入文件定义

- 1. 期权参数文件（文本格式）

代码块

```
1  option_type = european_call      # european_call / european_put / asian_call
   / barrier_call
2  spot = 100.0                    # 初始价格
3  strike = 100.0                  # 执行价格
4  risk_free_rate = 0.03           # 无风险利率
5  volatility = 0.2                # 波动率
6  maturity = 1.0                 # 到期时间，单位年
7  barrier = 130.0                # 障碍期权使用，可选
```

2. 模拟参数文件（文本格式）

代码块

```
1  num_paths = 100000000          # 模拟路径数
2  num_steps = 256                # 每条路径的时间步数
3  seed = 1234                    # 随机数种子
4  rng = "curand"                 # curand 或自定义 RNG
5  variance_reduction = "none"    # none / antithetic / control_variate
```

输出文件定义

1. **定价结果文件**：文本格式，包含期权类型、价格估计、参考价格、误差指标、收敛指标和使用的方法名称。
2. **性能日志**：包含总运行时间（ms）、路径生成速率（paths/sec）、粒度配置、归约时间、相比 CPU 单线程实现的加速比。

要求

1. 功能正确性：

- 至少支持欧式看涨期权和另一种路径依赖期权（亚式或障碍期权）；
- 欧式看涨期权需与 Black-Scholes 解析解对比，路径数增加时误差应收敛；
- CPU 版本和 CUDA 版本在相同/可比采样条件下结果应一致。

2. 方法设计与数值分析：

- 学员需自行调研并选择定价方法，报告中说明选择依据；
- 如使用确定性方法，需说明网格、边界条件、迭代收敛或截断误差；如使用随机方法，需说明随机数生成、置信区间和方差控制。

3. GPU 并行化：

- 主要计算需在 GPU 上完成；
- 需说明并行粒度、内存布局和 CPU/GPU 分工。

4. 进阶方向：

- 支持更多期权类型或风险指标（如 Delta、Gamma、Vega）；
- 对比两种以上数值方法或优化策略；
- 使用优化技巧进一步提升吞吐。

5. 平台适配：默认需在英伟达平台上支持。每在一款国产平台上支持，额外加分。

需提交内容：

- 整个完成上述功能的程序（包含测试）

- 提交地址：[Learning-CUDA 本季度项目分支 \(2026-summer-project\)](#)
 - 提交要求：参照本文档开头**项目阶段信息和简介-项目代码要求**中的**第四条**
 - 总结报告。报告需包括：
 - 详细阐述金融模型调研、定价方法选择、数值误差和 CUDA 并行化设计
 - 最终的正确性验证、收敛曲线、性能指标和分析
 - 方差缩减方法的效果分析（如实现）
 - 未来可继续提升的地方（可选）
 - 包含 ncu 和/或 nsys 使用和分析的加分
-

六. 贸易网络最大流量规划（CUDA）

任务背景

“天下熙熙皆为利来，天下攘攘皆为利往”。泛星系贸易市场的成立使得无数星球之间通过超空间航道连接，每条航道有固定的货运容量（万吨/天）。凭借你 InfiniTensor 训练营的经历和人工智能与并行计算方面的出色成绩，你被晋升为泛星系贸易管理局的算法官，负责实时计算从资源富集星球到需求中心的最大货物运输能力，为贸易调度和航道规划提供依据。

泛星系贸易市场体量巨大，星球数量已达百万，航道数量千万，传统 CPU 无法满足实时查询需求。你需要开发并行最大流计算程序，以闪电般的速度响应各星区的物流请求。如果效果显著，你的程序将被部署于国家级“星枢”计算阵列，直接影响国家的贸易，甚至决定一场星际饥荒的存亡。

领域知识简介

有向图（Directed Graph）是一种图结构，其中的边具有方向性。这意味着每条边都有一个起始节点和一个结束节点，表示数据/状态/进程从一个节点流向另一个节点。给定一个有向图，每条边有容量，求从源节点 `s` 到汇节点 `t` 能输送的最大流量。你需要“设计”一个并行友好的算法。

任务内容

开发一个 CUDA 程序，实现并行最大货物运输能力计算，处理单源单汇最大流，并输出规定的输出文件。

输入文件定义

1. **图文件（CSR 格式二进制）**：CSR（Compressed Sparse Row）格式存储有向图，包含三个数组：

- `row_ptr`：长度为 $N+1$ 的整数数组，`row_ptr[i]` 表示节点 `i` 的出边在 `col_indices` 和 `values` 中的起始索引，`row_ptr[N]` 为总边数 M 。
- `col_indices`：长度为 M 的整数数组，存储每条边的目标节点。

- `values`：长度为 M 的非负整数数组，存储每条边的容量。

示例：一个简单图：节点0→1(容量5)，节点0→2(容量3)，节点1→2(容量2)。则：

代码块

```
1 row_ptr = [0, 2, 3, 3]    // 节点0有2条边，节点1有1条，节点2有0条
2 col_indices = [1, 2, 2]    // 节点0的出边目标：1和2；节点1的出边目标：2
3 values = [5, 3, 2]        // 对应容量
```

文件存储为二进制，顺序：先写入 N (int)，M (int)，然后写入 `row_ptr` 数组 (N+1 个 int)，`col_indices` (M 个 int)，`values` (M 个 int)。

2. 源汇查询文件（文本格式）包含多个查询对，每行分别是： `source_id target_id`。

代码块

```
1 0 2
2 1 2
3 0 1
```

输出文件定义

1. 最大流结果文件：对于每个查询，输出一行 `source target maxflow`。
2. 性能日志：包含预处理时间、首次查询时间（TTFQ）、总查询时间（所有查询完成）、平均每查询时间（TPQ），可选输出算法内部统计信息。需说明统计是否包含文件读取、CPU-GPU 数据拷贝和残量网络初始化时间等。

要求

1. 图规模：图文件采用 CSR 格式，程序需支持节点数至少 10^5 ，边数至少 10^6 。初期测试可使用小图。
2. 数据类型：基础版本容量统一使用 `int32`，容量范围需覆盖 $1 \sim 10000$ ，结果使用 `int64` 输出。进阶可支持 `float` 容量，但需说明数值容差和验证方式。
3. 多查询支持：支持至少 10 个不同的源汇对查询。允许串行处理，但需统计总查询时间和平均每查询时间（TPQ）。多个查询相互独立，不得复用上一次查询后的残量状态作为下一次查询的初始状态。
4. 正确性验证：需至少与 CPU 串行实现在小图上对比，确保最大流值一致。
5. 性能指标：至少需统计：
 - 图预处理时间 `T_preprocess`
 - 首次查询完成时间 `TTFQ`

- 所有查询总时间 `T_total`
 - 平均查询时间 `TPQ`
6. **内存管理**：图拓扑和原始容量数据应常驻 GPU 显存。查询过程中不得因重新读取查询而重复传输图拓扑。每次查询可在 GPU 内重新初始化残量网络及算法工作数据。
7. **平台适配**：默认需在英伟达平台上支持。每在一款国产平台上支持，额外加分。

需提交内容：

- 整个完成上述功能的程序（包含测试）
 - 提交地址：[Learning-CUDA 本季度项目分支 \(2026-summer-project\)](#)
 - 提交要求：参照本文档开头**项目阶段信息和简介-项目代码要求**中的**第四条**
- 总结报告。报告需包括：
 - 详细阐述实现思路以及优化方法，欢迎包含自己的优化历程和记录，开发中发现的问题等（取决于质量可以加分）
 - 最终的性能指标和分析
 - 未来可继续提升的地方（可选）
 - 包含 `ncu` 和/或 `nsys` 使用和分析的加分

七. 实时图像非局部均值降噪（CUDA）

任务背景

鉴于你有 InfiniTensor 训练营的经历，并且在人工智能与并行计算方面的成绩出众，你被邀请加入“巡天”深空探测影像团队。探测器正在一颗远日小行星的背光面低空飞行，机载相机只能在极低照度和短曝光时间下拍摄。传回的画面充满随机噪声和传感器热噪声，而这些画面将直接用于地形重建、着陆点筛选和异常目标识别。

任务窗口非常短，探测器不能等待地面慢速处理后再调整姿态。普通均值滤波会抹掉陨石坑边缘和细小裂隙，过强的局部滤波又会保留大量颗粒噪声。你需要在机载 GPU 上实现一个实时图像降噪模块，在尽量保留纹理和边缘的同时抑制噪声，并给出图像质量与处理延迟之间的权衡分析。非局部均值（Non-Local Means, NLM）是一种可选且具有代表性的方案：它通过比较图像中小块 patch 的相似性来加权平均，但计算量极大。你的任务是实现一个可配置、可验证、可分析性能瓶颈的 GPU NLM 降噪程序。

领域知识简介

非局部均值降噪的核心思想是：自然图像中存在大量重复或相似结构，一个像素的真实值可以由图像中与其周围 patch 相似的多个像素加权得到。对于中心像素 `p` 和候选像素 `q`，权重通常由两个 patch 的距离决定：

```
代码块(p, q) = exp(-max(dist(P_p, P_q) - 2*sigma^2, 0) / h^2)
```

其中 `P_p` 和 `P_q` 是以 `p`、`q` 为中心的 patch，`sigma` 是噪声强度估计，`h` 是滤波强度。该算法的难点在于 patch 距离计算量大、边界处理复杂、全局内存访问重复。优化方向包括 shared memory 缓存、查表近似、分离通道处理、向量化访存和搜索窗口裁剪。

任务内容

开发一个 CUDA 程序，实现灰度图或 RGB 图像的非局部均值降噪。

输入文件定义

1. **图像文件**（标准格式，如 PNG、JPG），RGB 三通道或灰度。
2. **参数文件**（文本格式）

代码块

```
1 patch_radius = 3          # patch 半径, 实际 patch 大小为 (2*patch_radius+1)^2
2 search_radius = 10        # 搜索窗口半径
3 h = 10.0                  # 滤波强度
4 sigma = 25.0              # 噪声标准差估计, 对于8位图像范围0-255
```

输出文件定义

1. **降噪后的图像**：与输入同格式的文件。
2. **性能日志**：处理时间（ms）、吞吐量（百万像素/秒）、相比 CPU 实现或 OpenCV 参考实现的加速比。

要求

1. **图像格式**：输入图像支持灰度图或 RGB 彩色图，像素值范围为 0-255（整数），处理时转换为浮点。输出为同格式。
2. **滤波器参数**：`patch_radius` 至少支持到 3，`search_radius` 至少支持到 10，滤波强度 `h` 和噪声估计 `sigma` 可配置。
3. **正确性验证**：输出图像应与 CPU 参考实现或 OpenCV 的 `fastNlMeansDenoising` / `fastNlMeansDenoisingColored` 结果对比，计算平均绝对误差（MAE）或峰值信噪比（PSNR）。如采用近似算法，需说明误差来源。
4. **性能目标**：在至少 1920×1080 图像上进行测试，并给出不同参数组合下的运行时间。进阶目标为支持 4K 图像的交互式处理。
5. **平台适配**：默认需在英伟达平台上支持。每在一款国产平台上支持，额外加分。

需提交内容：

- 整个完成上述功能的程序（包含测试）
 - 提交地址：[Learning-CUDA 本季度项目分支 \(2026-summer-project\)](#)
 - 提交要求：参照本文档开头**项目阶段信息和简介-项目代码要求**中的**第四条**
- 总结报告。报告需包括：
 - 详细阐述实现思路以及优化方法，欢迎包含自己的优化历程和记录，开发中发现的问题等（取决于质量可以加分）
 - 最终的图像质量指标、性能指标和分析
 - 未来可继续提升的地方（可选）
 - 包含 ncu 和/或 nsys 使用和分析的加分

八. N 体引力模拟与可视化（CUDA）

任务背景

你是“巡天”深空探测项目的核心算法工程师，负责模拟小行星带、卫星碎片和探测器之间的引力相互作用，以辅助航迹规划和碰撞风险评估。地面控制中心每天需要评估大量不同初始条件下的演化结果，传统 CPU 程序在粒子数量上升后很快无法满足交互式分析需求。

你的任务是开发一个 GPU 加速的 N 体引力模拟器，能够并行计算数千到数十万个天体之间的相互作用，并输出轨迹数据供 Python 可视化。最终，你需要提供一个可以展示星团演化、轨道扰动和局部碰撞风险的动画脚本，帮助工程团队直观理解模拟结果。

领域知识简介

N 体问题研究多个粒子在相互作用力下的运动。对于引力系统，粒子 i 受到其他粒子 j 的作用：

代码块

```
1 F_ij = G * m_i * m_j * (r_j - r_i) / (|r_j - r_i|^2 + eps^2)^(3/2)
```

其中 G 是引力常数， m 是质量， r 是位置， eps 是软化因子，用于避免两个粒子距离过近时数值发散。直接算法复杂度为 $O(N^2)$ 。

任务内容

开发 CUDA 程序模拟 N 体引力系统，并输出轨迹点供 Python 可视化。任务包括：

1. 解析输入文件；
2. 使用并行 kernel 计算每个粒子的加速度，更新位置和速度；
3. 将轨迹数据传输回主机，保存为文件；

提供 Python 脚本读取数据并生成二维或三维动画。

输入文件定义

1. 粒子初始参数文件（文本格式）每行一个粒子，包含：

代码块

```
1   x y z vx vy vz mass
```

- `x, y, z`：初始位置
- `vx, vy, vz`：初始速度
- `mass`：粒子质量

示例：

代码块

```
1   0.0 0.0 0.0 0.0 0.0 0.0 1.0e6
2   1.0 0.0 0.0 0.0 1.0 0.0 1.0
3   0.0 1.0 0.0 -1.0 0.0 0.0 1.0
```

2. 模拟参数文件（文本格式）

代码块

```
1   dt = 1e-3           # 时间步长
2   num_steps = 1000     # 总步数
3   record_interval = 100 # 每多少步记录一次位置
4   G = 1.0             # 引力常数，可使用归一化单位
5   softening = 1e-4     # 软化因子
6   integrator = "leapfrog" # euler / leapfrog
```

输出文件定义

1. **轨迹数据文件**：二进制或 CSV 格式，每行包含 `particle_id, step, x, y, z`。
2. **性能日志**：包含模拟时间、粒子更新速率（particle-steps/sec）、每步平均耗时、显存占用和相比 CPU 实现的加速比。
3. **可视化脚本**：Python 脚本（`visualize.py`），使用 NumPy 读取轨迹数据，并使用 Matplotlib 的 `animation` 模块动态绘制轨迹。

要求

1. **粒子规模**：基础版本需支持粒子数至少 4096 个，总时间步数至少 1000 步。进阶版本支持 65536 个以上粒子。
2. **数值积分**：必须使用数值积分方法。
3. **正确性验证**：二体系统应能保持近似椭圆轨道或圆轨道；多体系统需验证动量或能量误差在合理范围内。
4. **轨迹存储**：轨迹数据存储为二进制文件，格式为：先写入粒子数 P 和记录点数 R (int)，然后按粒子顺序写入所有坐标 (float, 顺序为 x,y,z)。也可采用 CSV，但需说明效率较低。
5. **可视化脚本**：提供 Python 可视化脚本，使用 Matplotlib 绘制二维或三维动态轨迹，需包含动画效果 (`FuncAnimation`)。
6. **平台适配**：默认需在英伟达平台上支持。每在一款国产平台上支持，额外加分。

需提交内容：

- 整个完成上述功能的程序（包含测试）
 - 提交地址：[Learning-CUDA 本季度项目分支 \(2026-summer-project\)](#)
 - 提交要求：参照本文档开头**项目阶段信息和简介-项目代码要求**中的**第四条**
- 总结报告。报告需包括：
 - 详细阐述实现思路、数值积分方法和优化方法，欢迎包含自己的优化历程和记录，开发中发现的问题等（取决于质量可以加分）
 - 最终的正确性验证、性能指标和分析
 - 未来可继续提升的地方（可选）
 - 包含 ncu 和/或 nsys 使用和分析的加分

九. GPU 向量检索引擎（CUDA）

任务背景

凭借你 InfiniTensor 训练营的经历和人工智能与并行计算方面的出色成绩，你加入了一家大模型应用基础设施团队。团队正在建设面向 RAG、推荐系统和多模态检索的向量数据库服务。每一次用户查询都会被编码成高维向量，并需要在数百万到数千万条候选向量中找出最相似的 Top-K 结果。

CPU 实现虽然容易开发，但在高并发、低延迟场景下很快遇到瓶颈。GPU 有强大的并行计算和内存带宽，但向量检索并不是简单地写一个距离计算 kernel：你需要考虑数据布局、索引结构、Top-K 归并、批处理大小、显存容量、召回率和延迟之间的权衡。你的任务是开发一个 GPU 加速的向量检索引擎，并用实验说明自己的系统设计取舍。

领域知识简介

向量检索的目标是：给定查询向量 q ，在向量库 $\{x_i\}$ 中找到距离最近或相似度最高的 K 个向量。常见度量包括 L2 距离、内积和 cosine similarity。精确检索需要计算所有候选向量与查询向量的距离，复杂度为 $O(ND)$ ，其中 N 是向量数量， D 是维度。

在大规模系统中，通常会使用近似最近邻（Approximate Nearest Neighbor, ANN）方法，例如 IVF-Flat、IVF-PQ、HNSW 等。

任务内容

开发一个 CUDA 程序，实现一个简化但完整的 GPU 向量检索引擎。程序需要支持精确检索 baseline，并至少实现一种近似检索方案或索引优化方案。

输入文件定义

1. 向量库文件（二进制格式）

代码块

```
1  [header]
2  num_vectors: int64           # 向量库规模
3  dim: int32                   # 向量维度
4  dtype: string                # fp32 或 fp16
5  metric: string               # l2 / inner_product / cosine
6
7  [data]
8  vectors: dtype[num_vectors * dim] # 按行主序存储
```

2. 查询向量文件（二进制格式）

代码块

```
1  [header]
2  num_queries: int64
3  dim: int32
4  dtype: string
5
6  [data]
7  queries: dtype[num_queries * dim]
```

3. 检索参数文件（文本格式）

代码块

```
1  top_k = 10
2  search_mode = "exact"           # exact / ivf_flat / ivf_pq
3  batch_size = 128                # 查询批大小
```

4	nlist = 4096	# IVF 聚类中心数量
5	nprobe = 16	# 查询时访问的倒排桶数量
6	pq_m = 16	# PQ 子空间数量，可选

输出文件定义

1. **检索结果文件**：每个 query 输出 Top-K 的向量 id 和距离 / 相似度。
2. **索引文件**：如实现近似检索，需保存索引元数据、聚类中心、倒排表或压缩码本。
3. **性能日志**：包含建索引时间、查询总时间、QPS、P50 / P99 延迟、显存占用、相比 CPU baseline 的加速比。
4. **质量日志**：包含 recall@K、平均距离误差，以及不同 `nprobe` / batch size 下的性能和召回率变化。

要求

1. 功能正确性：

- 必须实现 GPU 精确检索 baseline，支持 L2 距离或 inner product 中至少一种；
- 精确检索结果需与 CPU 参考实现一致；
- Top-K 输出需按距离或相似度排序；
- 支持批量查询，不能只支持单 query。

2. 索引与近似检索：

- 至少实现 IVF-Flat、IVF-PQ 或其他一种明确说明的近似检索/索引优化方案；
- 需支持从文件加载已构建索引，避免每次查询都重新建索引；
- 需与精确检索 baseline 对比 recall@K。

3. Top-K 与归并：

- 需支持 `K = 1, 10, 50, 100` 中至少三种。

4. 数据规模：

- 基础测试需支持至少 `10^6` 条向量、维度至少 128、查询数至少 1000；
- 初期开发可使用更小规模数据；

5. 平台适配：默认需在英伟达平台上支持。每在一款国产平台上支持，额外加分。

需提交内容：

- 整个完成上述功能的程序（包含测试）
 - 提交地址：[Learning-CUDA 本季度项目分支 \(2026-summer-project\)](#)
 - 提交要求：参照本文档开头[项目阶段信息和简介-项目代码要求](#)中的[第四条](#)

- 总结报告。报告需包括：
 - 详细阐述检索算法、索引结构、数据布局和优化方法，欢迎包含自己的优化历程和记录，开发中发现的问题等（取决于质量可以加分）
 - 与 CPU 精确检索或 FAISS 等参考实现的正确性和召回率对比
 - 最终的 QPS、P50 / P99 延迟、recall@K、显存占用和性能分析
 - 未来可继续提升的地方（可选）
 - 包含 ncu 和/或 nsys 使用和分析的加分
-